Pion Soft Factor Measurement

A Detailed Physics Note for the PyQUDA Implementation

October 27, 2026

1 Physics Overview

The pion soft factor is a four-point correlation function that appears ubiquitously in the factorization of transverse-momentum-dependent (qTMD) observables. In the Collins–Soper (CS) framework [1, 2] and the Ji–Ma–Yuan (JMY) framework [3, 4], the soft factor absorbs rapidity divergences that arise when one attempts to separate the hard scattering, the collinear beam functions, and the soft radiation in hadron scattering processes.

For a pion, the soft factor is defined through a matrix element of four Wilson-line staple operators arranged on the light-cone. In a Euclidean lattice calculation one gives up the light-cone separation and instead works with equal-time correlators in a large-momentum frame, following the LaMET/quasi-PDF philosophy. The Lorentz invariant definition is then replaced by a quasi-soft-factor whose infrared structure coincides with the physical soft factor after a perturbative matching.

Concretely, the lattice object we compute is

$$S(b_\perp, t; t_0, t_{\text{sep}}, \Gamma_1, \Gamma_2) \;=\; \sum_{\mathbf{x}} \text{Tr}\left[A_b(\mathbf{x}, t)\, \Gamma_2\, B_b(\mathbf{x}, t)\, \Gamma_1 \right], \tag{1}$$

where $b_\perp$ is a transverse displacement between the quark and antiquark Wilson lines, t_0 is the source time slice, t_{sep} is the sink–source separation, and the closed spin-color blocks A_b and B_b are built from four Coulomb-gauge-fixed wall-source propagators as described in detail below. The matrices Γ_1 and Γ_2 are drawn from a reduced gamma set appropriate for the soft factor.

The physical interpretation of Eq. (1) is that the two blocks A_b and B_b represent the quark and antiquark lines that travel from the source to the sink, with the transverse displacement b inserted to probe the transverse-momentum dependence of the soft radiation. The trace over spin and color indices projects onto the desired gamma structures.

2 Two-Stage Workflow

The calculation is intentionally split into two independent stages to allow flexible reuse of the expensive propagator inversions.

Stage 1 – Propagator Generation (`Pyquda_pion_soft_factor_prop.py`)

- Read a Coulomb-gauge-fixed configuration.
- For every requested source time slice t_0 and every quark momentum $\mathbf{k}$ (and $-\mathbf{k}$), construct a momentum-phase wall source and invert the Dirac operator.
- Save each propagator as an HDF5 file with full metadata (lattice tag, configuration number, smearing tag, source position, quark momentum).

Stage 2 – Contraction (`Pyquda_pion_soft_factor_contract.py`)

- Load the wall propagators saved in Stage 1 (no gauge field needed except for lattice geometry metadata).
- For each pair of forward/backward quark momenta $(\mathbf{k}_{\rm fw}, \mathbf{k}_{\rm bw})$ compute:
  (i) the wall-to-wall two-point diagnostic $C_2(t)$,
  (ii) the qTMDWF-like diagnostic $C_{\rm TMDWF}(b_T, b_z, t)$,
  (iii) the full soft-factor four-point correlator $S(b_\perp, t; t_{\rm sep}, \Gamma_1, \Gamma_2)$.
- Save all correlators to HDF5 with appropriate group hierarchies.

This two-stage design means that one can change contraction parameters (e.g. Γ_1, Γ_2, b_T ranges, sink interpolators) without re-running any inversions.

3 Wall-Source Propagators

3.1 Coulomb-Gauge Wall Source

The source for quark momentum $\mathbf{k}$ at time t_0 is

$$\eta_{\mathbf{k}}(\mathbf{x}, t) \;=\; \delta_{t,\,t_0}\; \exp\big(+\,i\,\mathbf{k}\cdot\mathbf{x}\big)\,. \tag{2}$$

The spatial part is a plane wave with momentum $+\mathbf{k}$, applied uniformly on every spatial site of time slice t_0. The Coulomb gauge condition is imposed before the wall source is constructed, which suppresses the gauge noise that would otherwise contaminate a point source. The corresponding propagator is

$$G_{\mathbf{k}}(x; t_0) \;=\; D^{-1}\,\eta_{\mathbf{k}}\,, \tag{3}$$

where D is the Wilson-clover Dirac operator with appropriate boundary conditions (t-boundary set to antiperiodic, `t_boundary = -1`).

3.2 Both Signs of Quark Momentum

The soft-factor contraction requires both $+\mathbf{k}$ and $-\mathbf{k}$ propagators on all time slices. Stage 1 therefore loops over a set of momenta that includes both signs:

$$\mathcal{K}_{\rm save} \;=\; \{\,\mathbf{k}_{\rm fw},\, \mathbf{k}_{\rm bw},\, -\mathbf{k}_{\rm fw},\, -\mathbf{k}_{\rm bw}\,\} \quad \text{(duplicates removed)}. \tag{4}$$

3.3 Antiquark Line from Gamma5 Hermiticity

The Euclidean Dirac operator satisfies γ_5-hermiticity,

$$D^\dagger \;=\; \gamma_5\, D\, \gamma_5\,. \tag{5}$$

For an antiquark propagating *backward* in time the required propagator is obtained from the $-\mathbf{k}$ forward propagator via

$$\bar{G}_{-\mathbf{k}}(x; t_0) \;=\; \gamma_5\, G_{-\mathbf{k}}(x; t_0)^\dagger\, \gamma_5\,. \tag{6}$$

The dagger exchanges both spin and color ends. For a field stored as `tzyxmlca`, an explicit reference implementation is

```
xp.einsum("kl,tzyxmlca,mj->tzyxkjac",
          gamma5, prop.conj(), gamma5)
```

The production implementation packs row (spin, color) and column (spin, color) into 12×12 matrices and uses ordinary matrix adjoints. Color indices are not spectators to the transpose. The historical `ij,tzyxmlca,kl` expression summed unrelated spin indices and is not used.

3.4 Momentum Phase Convention

The phase convention is tied to the definition of the wall source, Eq. (2):

Operation	Phase factor
Wall source at t_0 with momentum $\mathbf{k}$	$\exp(+i\,\mathbf{k}\cdot\mathbf{x})$
SP phase on G_{fw}	$\exp(-i\,\mathbf{p}\cdot\mathbf{x})$ where $\mathbf{p}$ is the meson momentum
Sink-side phase on $G_{\mathbf{k}_{\mathrm{bw}}}$ (soft factor)	$\exp(-2i\,\mathbf{P}\cdot\mathbf{x})$ where $\mathbf{P}=\mathbf{k}_{\mathrm{fw}}+\mathbf{k}_{\mathrm{bw}}$

The sign of the applied phase in `apply_phase` is controlled by the `sign` argument. When `sign=1`, the raw momentum-phase factor $\exp(+i\mathbf{k}\cdot\mathbf{x})$ is multiplied onto the propagator data. When `sign=-1`, the complex conjugate $\exp(-i\mathbf{k}\cdot\mathbf{x})$ is used instead.

4 Wall-to-Wall Two-Point Diagnostic

The SS diagnostic follows the native GPT implementation. Each line is projected and spatially summed separately, with no volume normalization:

$$F(t)=\sum_{\mathbf{y}} e^{-i\mathbf{k}_{\mathrm{fw}}\cdot\mathbf{y}}G_{\mathrm{fw}}(\mathbf{y},t), \tag{7}$$

$$B(t)=\sum_{\mathbf{x}} e^{+i\mathbf{k}_{\mathrm{bw}}\cdot\mathbf{x}}G_{\mathrm{bw}}(\mathbf{x},t), \tag{8}$$

$$C_2^{\mathrm{SS}}(t)=\mathrm{Tr}\Big[\Gamma_{\mathrm{src}}\gamma_5 B(t)^\dagger\gamma_5\Gamma_{\mathrm{sink}}F(t)\Big]. \tag{9}$$

The momentum on the backward forward-propagating line is $-\mathbf{k}_{\mathrm{bw}}$; the extra positive phase above becomes negative upon taking the dagger. The routine takes both quark momenta explicitly, rather than attempting to recover them from their sum.

The open matrices F and B are summed globally before multiplication. Tracing rank-local products and then reducing would lose products between different spatial MPI subdomains. One root-only reduction collects both open lines in global time order; rank zero then performs the small matrix trace.

5 qTMDWF Diagnostic

The SP diagnostic uses the full 16-element canonical raw sink Gamma basis:

$$C_{\mathrm{SP}}^{g}(\mathbf{b},t)=\sum_{\mathbf{x}} e^{-i\mathbf{p}\cdot\mathbf{x}}\,\mathrm{Tr}\Big[\Gamma_{\mathrm{src}}\gamma_5 G_{\mathrm{bw}}(\mathbf{x}+\mathbf{b},t)^\dagger\gamma_5\Gamma_g G_{\mathrm{fw}}(\mathbf{x},t)\Big],\qquad \mathbf{p}=\mathbf{k}_{\mathrm{fw}}+\mathbf{k}_{\mathrm{bw}}. \tag{10}$$

Here $\mathbf{b}=b_T\hat{n}_T+b_z\hat{z}$. The phase is evaluated at the unshifted forward coordinate. The source matrix is the first member of the selected pion pair; the sink scans the shared 16-Gamma basis independently of the second member, which is used by SS and C4.

The return shape is `[sink_gamma,separation,time]`, with separation flattened in direction, b_T, b_z order. This is the native GPT `slice_trDA` definition after cyclically rotating the trace. The Identity sink is index 9 (zero based). Raw PyQUDA `Y5` and `T5` require the stored `physical_from_pyquda` signs to match GPT's uniform $\gamma_\mu\gamma_5$ convention.

SS and SP are different observables: SS has independent sums over $\mathbf{x}$ and $\mathbf{y}$, whereas SP multiplies the lines at the same coordinate before summing. Their zero-displacement values need not agree. The former nine-component TMDWF compression relied on disconnected spin sums and has been removed; all shifted fields now retain the full spin-color structure.

6 Soft-Factor Four-Point Contraction

This is the main measurement. For each source time t_0 and sink separation t_{sep} the contraction uses four Coulomb-gauge wall propagators, organized by their role in the forward/backward quark and antiquark lines:

$$\begin{aligned} G_w &\equiv G_{+\mathbf{k}_{\text{fw}}}(t_0) \qquad \text{(forward quark, source time)} \\ G^{\dagger}_{w,\mathrm{b}_\perp} &\equiv G_{-\mathbf{k}_{\text{bw}}}(t_0) \qquad \text{(backward antiquark, source time)} \\ G^{\dagger}_{w} &\equiv G_{-\mathbf{k}_{\text{fw}}}(t_0 + t_{\text{sep}}) \qquad \text{(forward antiquark, sink time)} \\ G^{\mathrm{b}_\perp}_{w} &\equiv G_{+\mathbf{k}_{\text{bw}}}(t_0 + t_{\text{sep}}) \qquad \text{(backward quark, sink time)} \end{aligned} \tag{11}$$

The naming convention follows the legacy code:

- `Gw`: **G**auge-fixed **w**all propagator for the forward quark at the source.
- `Gw_bperp_dagger`: The dagger emphasizes that this propagator will be gamma5-conjugated ("dagger") and that it corresponds to the backward line that will be transversely displaced ("bperp").
- `Gw_dagger`: Forward antiquark at the sink, gamma5-conjugated.
- `Gw_bperp`: Backward quark at the sink, transversely displaced.

6.1 Sink-Side Momentum Phase

The backward quark propagator at the sink, $G^{\mathrm{b}_\perp}_w$, is multiplied by the momentum-transfer phase

$$G^{\mathrm{b}_\perp}_w \;\longrightarrow\; \exp\!\big(-2i\,\mathbf{P}\cdot\mathbf{x}\big)\; G^{\mathrm{b}_\perp}_w, \qquad \mathbf{P} \equiv \mathbf{k}_{\text{fw}} + \mathbf{k}_{\text{bw}} \tag{12}$$

where $\mathbf{P}$ is the total pion momentum. The factor of 2 arises from the kinematic structure of the soft factor, where both the source and sink sides contribute equal and opposite phases to the four-point function. All other propagators carry their momentum through the wall-source construction; only $G^{\mathrm{b}_\perp}_w$ receives this explicit phase because it is the only propagator for which the wall-source momentum ($+\mathbf{k}_{\text{bw}}$) does not match the required total momentum projection.

6.2 Time-Major Lexicographic Layout

Before contraction, all four propagators are rearranged into *time-major* lexicographic order via

```
prop.lexico(False)
```

This yields the index ordering

$$\underbrace{t}_{\text{time}}\ \underbrace{z}_{\text{slowest spatial}}\ \underbrace{y}_{\text{middle spatial}}\ \underbrace{x}_{\text{fastest spatial}}\ \underbrace{j,\,i}_{\text{spin sink, source}}\ \underbrace{b,\,a}_{\text{color sink, source}} \tag{13}$$

or, in the einsum convention used throughout the code,

```
tzyx j i b a
```

(14)

where Latin letters (i, j, k, l) denote Dirac spin indices and (a, b, c) denote color indices. This time-major layout is essential because the final trace over spatial volume and the time projection must respect the lexicographic order for correct MPI halo exchanges.

7 Closed Spin-Color Blocks

For each transverse displacement vector $\mathbf{b}$ (pointing along the axis $\hat{n}_{\mathrm{T}}$ with magnitude b_T lattice units), we form two closed spin-color blocks:

7.1 Block A_b – Source Side

$$A_b(\mathbf{x},t) \;=\; G_w(\mathbf{x},t) \;\cdot\; \Gamma_{\text{src}} \;\cdot\; \gamma_5 \;\cdot\; G_w^{\text{b}_\perp\dagger}(\mathbf{x}+\mathbf{b},\,t)^\dagger \cdot\; \gamma_5\,. \tag{15}$$

In words:

1. Start with the forward quark propagator G_w at position $\mathbf{x}$.
2. Insert the source interpolator Γ_{src}.
3. Gamma5-conjugate the transversely-shifted backward antiquark propagator $G_w^{\text{b}_\perp\dagger}$ (the dagger in the name refers to the fact that this propagator is stored as $G_{-\mathbf{k}_{\text{bw}}}$ and must be conjugated).
4. The shift by $+\mathbf{b}$ is applied to $G_w^{\text{b}_\perp\dagger}$ before conjugation.

7.2 Block B_b – Sink Side

$$B_b(\mathbf{x},t) \;=\; G_w^{\text{b}_\perp}(\mathbf{x}+\mathbf{b},\,t) \cdot\; \Gamma_{\text{sink}} \cdot\; \gamma_5 \;\cdot\; G_w^{\dagger}(\mathbf{x},\,t)^\dagger \cdot\; \gamma_5\,. \tag{16}$$

In words:

1. Start with the transversely-shifted backward quark propagator $G_w^{\text{b}_\perp}$ at position $\mathbf{x}+\mathbf{b}$. This propagator already carries the sink-side phase, Eq. (12).
2. Insert the sink interpolator Γ_{sink}.
3. Gamma5-conjugate the forward antiquark propagator $G_w^\dagger$ at position $\mathbf{x}$.

Both blocks A_b and B_b are matrices in spin-color space at each spacetime point $(\mathbf{x},t)$. Their open indices are the spin and color indices that will be contracted in the final trace.

7.3 Four-Point Correlator

With the blocks defined, the soft-factor four-point function is assembled as

$$C_4(t;t_{\text{sep}},\mathbf{b},\Gamma_1,\Gamma_2) \;=\; \sum_{\mathbf{x}} \text{Tr}\left[A_b(\mathbf{x},t)\;\Gamma_2\;B_b(\mathbf{x},t)\;\Gamma_1\right]. \tag{17}$$

The trace is over the spin and color indices that connect A_b and B_b; the spatial sum $\sum_{\mathbf{x}}$ projects onto zero (or total) spatial momentum after the phase factors have been accounted for in the propagator definitions.

8 Gamma and Interpolator Conventions

8.1 Soft-Factor Gamma Set

The soft factor uses a reduced set of gamma structures compared to standard meson spectroscopy. The PyQUDA gamma matrices follow the Euclidean conventions of the underlying library:

Key	Matrix	Description
`"5"`	$\gamma_5 = \gamma_5$	Chirality
`"I"`	$\mathbf{1}$	Identity
`"X"`	γ_1	γ_1
`"Y"`	γ_2	γ_2
`"X5"`	$\gamma_1\gamma_5$	$\gamma_1\gamma_5$
`"Y5"`	$-\gamma_2\gamma_5$	Raw PyQUDA axial channel

In the code these are stored in the dictionary `_raw_gamma_by_label`, which maps each key to the corresponding PyQUDA `gamma` object. The `gamma(15)` denotes γ_5 (the 16th element in the standard Euclidean gamma basis, 0-indexed), and `gamma(0)` is the identity.

8.2 Pion Interpolators

The default pion interpolator set is

$$\texttt{"Z5-X5"} \;=\; \gamma_3\gamma_5 - \gamma_1\gamma_5 \;=\; \gamma_3\gamma_5 - \gamma_1\gamma_5\,. \tag{18}$$

This is a linear combination that projects onto the pseudoscalar pion with good overlap at finite momentum. Both Γ_1/Γ_2 (the soft-factor gamma insertions) and `pion_src`/`pion_sink` (the pion interpolators) are passed as Python dictionaries, so the user can supply arbitrary operator sets.

The source and sink matrices are the two members of each explicit pion channel pair; insertion channels likewise retain ordered Gamma1/Gamma2 pairs.

8.3 Gamma5 as γ_5

The matrix γ_5 appears in three distinct roles:

1. **Gamma5-hermiticity conjugation** of antiquark propagators, Eq. (6).
2. **Block closure** in A_b and B_b, Eqs. (15)–(16).
3. **As a gamma insertion**: when Γ_1 or Γ_2 is `"5"`, it acts as a genuine operator insertion probing the chiral structure of the soft factor.

In the code, the variable `G5 = gamma.gamma(15)` is used for roles 1 and 2, while role 3 uses the entry `_raw_gamma_by_label["5"]`, which is the same underlying matrix.

9 Transverse Displacement Implementation

Positive displacement reads the next coordinate: $G^{[b]}(x) = G(x + b\hat{n})$, the convention of native GPT `cshift` and PyQUDA `shift`. On a single spatial partition this agrees with a *negative* NumPy/CuPy roll. The historical positive roll has the opposite sign.

The canonical `_shifted_lexico_array` walker advances one site at a time using reusable buffers. For a distributed direction it sends the first face to the minus neighbor and receives the plus neighbor's first face into the last face. Thus periodic wrapping uses the global lattice, not the local subdomain. The physics directions x, y, z map to lexicographic axes $3, 2, 1$. Nested transverse and longitudinal walks own separate buffers; callers consume each temporary view before advancing its walk.

`bT_dir` selects the transverse directions, `bT_length` is the inclusive transverse maximum, and `bz_length` is the inclusive longitudinal maximum for SP only.

10 Code Implementation

The core logic lives in the class `pion_soft_factor` (file `pion_soft_factor_vibe_develop.py`). This section documents each method.

10.1 `__init__(parameters)`

Accepts a dictionary with keys:

Key	Description
`quark_mom`	List of quark 3-momenta $[\mathbf{k}_1, \mathbf{k}_2, \dots]$
`bT_dir`	List of transverse displacement axes
`bT_length`	Maximum b_T
`bz_length`	Maximum b_z (qTMDWF diagnostic)
`tsep_list`	List of sink separations
`pion_channel_pairs`	Ordered pair label $\mapsto (\Gamma_{\text{src}}, \Gamma_{\text{sink}})$
`gamma_channel_pairs`	Ordered pair label $\mapsto (\Gamma_1$ label, Γ_2 label)

The C4 channel axes are paired rather than Cartesian products. The stored order is

```
[pion_pair, gamma_pair, bT_direction, bT, time]
```

with an outer `tsep` axis in the combined output. Each pair label and both members are written explicitly to HDF5. The insertion labels are resolved through the canonical raw PyQUDA basis `5,I,X,Y,X5,Y5`; in particular `Y5` is the raw bit-mask matrix, not a separately sign-adjusted physical axial matrix. The file stores the raw basis, PyQUDA IDs, and `physical_from_pyquda` transform.

10.2 `create_wall_src(latt_info, tslice, momentum)`

Constructs the Coulomb-gauge wall source, Eq. (2). Internally it calls `phase.MomentumPhase` to build the momentum phase factor $\exp(+i\mathbf{k}\cdot\mathbf{x})$ and `source.propagator` to place it on the specified time slice.

10.3 `create_wall_propagator(dirac, latt_info, tslice, momentum)`

Assembles the wall source and inverts the Dirac operator: `core.invertPropagator(dirac, wall_src, 1, 0)`. The arguments `1, 0` specify the multigrid level and the number of deflation vectors, respectively (standard single-level inverter with no deflation).

10.4 `save_wall_propagator(prop, tag, attrs=None)`

Saves the propagator to HDF5 using `prop.saveH5(tag + ".h5", "propagator")`. Optional metadata (lattice tag, configuration number, smearing, source time, quark momentum) is written as HDF5 attributes for reproducibility.

10.5 `load_wall_propagator(tag)`

Loads a previously saved propagator via

```
core.LatticePropagator.loadH5(tag + ".h5", "propagator")
```

10.6 `apply_phase(prop, momentum, sign=1, x0=None)`

Multiplies the propagator data by a momentum phase factor:

- Computes the phase $\exp(\pm i\mathbf{k}\cdot(\mathbf{x}-\mathbf{x}_0))$ using `MomentumPhase.getPhase`.
- `sign=1` gives the positive phase; `sign=-1` gives the complex conjugate.
- The phase array is broadcast over the spin-color trailing dimensions.
- Returns a copy of the propagator with the phased data.

10.7 SS and SP contractions

`contract_wall_2pt` takes lattice info, two propagators, both quark momenta and a pion pair label, and returns the global-time SS array on rank zero. The routine phases and sums each line separately, globally reduces the open matrices, and only then forms the full trace in Eq. (9).

`contract_tmdwf_check` takes lattice info, two propagators, the total pion momentum and a pion pair label. It prepares the forward/source operand once, walks the full backward field through all separations, and contracts against all 16 sink matrices. One root-only reduction returns `[16,n_dir*(n_bT+1)*(n_bz+1),Lt]`. Non-root ranks return `None` in both routines. The application rolls the final time axis by $-t_0$ before writing the result.

10.8 `contract_soft_factor`

The main four-point contraction, described in full in Sec. 11. Returns the five-dimensional array `(n_src, n_gm1, n_dir, n_bT+1, Lt)` together with the ordered key lists.

11 Matrix Blocks and Trace Contractions

The lexicographic field order is

$$G[t, z, y, x, j, i, b, a], \tag{19}$$

where (j, b) are the matrix row and (i, a) the column. Packing into a 12×12 matrix interleaves spin and color; every spin insertion is $\Gamma \otimes I_3$. Both block builders implement

$$M(L, S, R) = L(S \otimes I_3)(\gamma_5 \otimes I_3)R^\dagger(\gamma_5 \otimes I_3). \tag{20}$$

Source and sink preparation reuse fixed operands. The transpose applies to the conjugated propagator before right multiplication by gamma5, so the implementation does not assume a symmetric gamma5 matrix.

For an independent index expansion of either block:

```
"tzyxjiba,sik,kl,tzyxmlca,mn->stzyxjnbc"
```

The operands are L, the source/sink matrix stack, gamma5, R^* and gamma5. Both outputs have normal matrix row/column order. Closing the C4 color trace therefore requires opposite color labels in the two blocks:

```
color_spin = xp.einsum("tzyxjiba,tzyxklab->tjikl", A, B)
corr = xp.einsum("tjikl,gik,glj->gt", color_spin, Gamma2, Gamma1)
```

The repeated `ba,ba` ordering in the historical implementation instead transposed only the color part of the second block and is incorrect. The SP projection of $A = G_{\mathrm{fw}}\Gamma_{\mathrm{src}}\bar{G}_{\mathrm{bw}}$ is

```
xp.einsum("tzyxjiaa,gij->gt", A, sink_gammas)
```

The repeated color label explicitly takes the color diagonal; the spin labels close through the sink matrix. Spatial sums precede one batched root-only reduction for SP and C4.

12 HDF5 Output Structure

Propagator files are unchanged and can be reused for corrected contractions. Every new correlator file records the following metadata:

- `soft_factor_schema`: `full_spin_color_v3`.
- `correlator_definition` and axis names.
- The positive-shift convention and canonical raw basis metadata.

- `physical_from_pyquda` for the Gamma convention conversion.

Old correlator files do not satisfy this schema and must be recontracted; there is no legacy contraction mode or duplicate output hierarchy.

```
SS:
  /SS/pion_pair/{pair}/PX{px}PY{py}PZ{pz}
SP:
  /SP/pion_pair/{pair}/PX{px}PY{py}PZ{pz}/gamma/{sink}/b_X/bT{b}/bz{z}
C4:
  /pion_pair/{pair}/gamma_pair/{gpair}/bX_{b}/ts{tsep}
```

Each leaf is one complex global-time array, already rolled by the application. SP writes all 16 sink labels in `sink_gamma_labels`; transverse directions also include `b_Y`/`b_Z`. SS records both `quark_momentum_fw` and `quark_momentum_bw`; its displayed pion momentum is their sum. C4 retains the paired source/sink and insertion labels and explicit source/sink matrices.

13 Comparison with Native GPT and Validation

The reference is the native GPT matrix expression $\sum_x \mathrm{Tr}[A\Gamma_2 B\Gamma_1]$, not the historical explicit-einsum script. SS and SP use the native GPT spatial-sum definitions. Gamma labels use the repository's shared canonical raw basis; apply its stored transform before comparing with GPT's uniform axial convention. Fixed gauge preprocessing, wall-source momenta, phase origin, shift sign and source time must also match.

`tests/soft_factor_reference.py` independently copies each spin/color component into matrix rows and columns and uses NumPy matrix products, adjoints and traces. It does not call production packing, block, einsum or halo routines. Tests cover random complex inputs and interpolators, all 16 SP sinks, ordered C4 insertions, nonzero phases, asymmetric volumes, positive shifts and wraps, input immutability, SS cross-site/cross-rank terms, root ownership, HDF5 channel identity and raw-to-physical Gamma signs. Exact norm and off-diagonal color examples catch the former disconnected-spin and color-transpose errors. CPU halo fixtures check neighbor direction and boundary faces; they do not replace a real distributed GPU production rerun.

Acknowledgments

This work builds on the PyQUDA framework and the legacy GPT/PyQUDA mixed soft-factor code. The implementation follows the conventions established in the pion qTMD and qTMDWF analyses.

References

[1] J. C. Collins and D. E. Soper, "Back-To-Back Jets in QCD," Nucl. Phys. B **193**, 381 (1981).

[2] J. C. Collins and D. E. Soper, "The Theorems of Perturbative QCD," Ann. Rev. Nucl. Part. Sci. **37**, 383 (1987).

[3] X. Ji, J. Ma and F. Yuan, "QCD factorization for semi-inclusive deep-inelastic scattering at low transverse momentum," Phys. Rev. D **71**, 034005 (2005) [arXiv:hep-ph/0404183].

[4] X. Ji, Y. Liu and Y.-S. Liu, "Transverse-momentum-dependent parton distribution functions from large-momentum effective theory," Phys. Lett. B **811**, 135946 (2020) [arXiv:1911.03840 [hep-ph]].